



Mission Orientated Flight Software Development: CubeSTEP

Tirth Thakkar, Howard Lee, Viren Kumar, Maranda Laws,
Shridhi Seth, Marco Maggia, Navid Nakhjiri

California Polytechnic University, Pomona



Introduction

Small satellite teams often develop flight software and mission operations separately, leading to software behaviors that drift out of sync with ground procedures. To fix this, we present a co-design approach that directly links mission goals and operational procedures to the onboard flight software architecture. Demonstrated on the CubeSTEP-Cerberus mission using NASA's F' framework and FreeRTOS, our system maps mission intent into five constrained software states (Initialization, Safe, Idle, Experiment, and Transmit) guarded by automated health checks. This co-design strategy ensures the spacecraft acts autonomously to preserve its goals while strictly maintaining operator control over critical decisions.

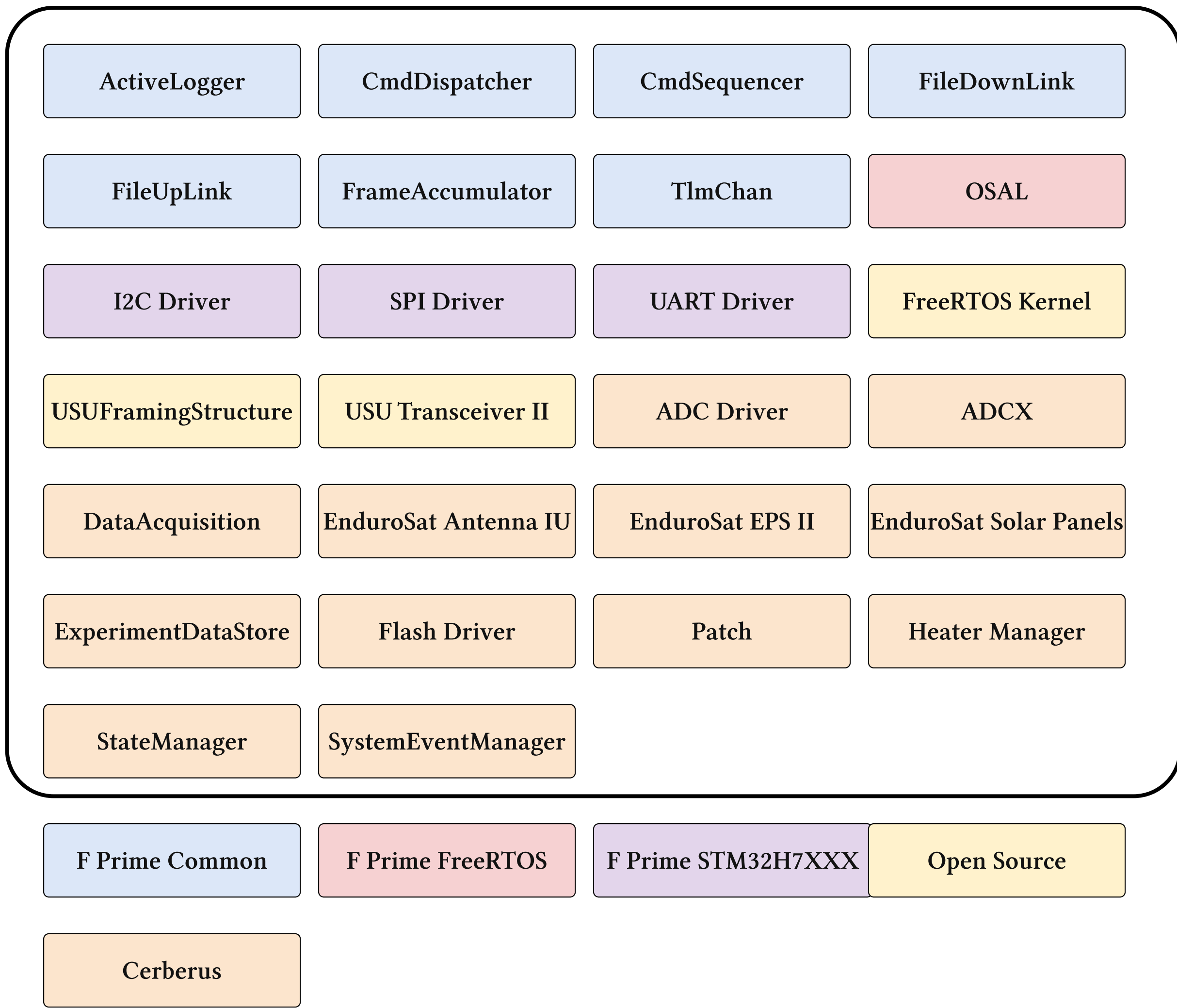
CONOPS

Concept of Operations are focused on Survivability, Communication, Data Collection, and Experiment Execution in that order.

Mission constraint	Software rule or guard
Orbital phase affects experiment	Experiment entry requires the requested sunlight condition or ap- validity
Intermittent communication	Spacecraft must maintain health, beacon status, and queue data with- out ground contact.
Limited power margin	Heater activity and high-rate downlink are inhibited below EPS thresholds.
Limited storage and downlink	Experiment data cannot overwrite undownlinked data without an approved rule.
Commissioning before science	Payload execution is locked out until required bus and sensor checks pass.
Decommissioning is irreversible	Final payload disablement and data closeout require ground autho- rization.

This structure preserves execution intention while being actionable. Integrating these rules into general systems allows us to create an behavioral logic framework that can robustly transition into software

Flight Software Implementation



Mission Software

Tools and Methodology

Cerberus uses F Prime with FreeRTOS on an STM32H7 onboard computer. A reusable support layer integrates F Prime with FreeRTOS, CMSIS, ST HAL, startup code, linker configuration, and peripheral drivers while keeping mission software independent of the selected board.

F Prime Common Services

F Prime services provide command dispatch, sequencing, telemetry, event logging, communication framing, and file transfer. These services form the spacecraft command and data-handling path between the ground system, flight components, radio, and onboard storage.

FreeRTOS and STM32 Platform Support

FreeRTOS schedules command processing, health monitoring, telemetry, payload sampling, and storage operations. Priority-based execution allows safety and health functions to preempt payload and communication activities when spacecraft conditions degrade.

Cerberus Mission Components

StateManager controls transitions among Initialization, Safe, Idle, Experiment, and Transmit, while SystemEventManager enforces CONOPS-derived health and readiness rules. Mission components manage heater control, data acquisition, flash storage, and interfaces to the EPS, solar panels, antenna, transceiver, ADCSX, and payload electronics. Ground-authorized software patches are permitted only in Idle after validation against mission intent.

Testing and Verification

Verification includes GTest unit tests, deployment on processor-representative and flight hardware, and subsystem integration testing in Linux and embedded environments. Testing validates state behavior, hardware interfaces, telemetry, storage, packetization, transmission, resource use, and assertion-free operation under nominal and off-nominal conditions. **Initialization state** is entered after deploying from the launcher. Detumbling executes until reaching stable body rates, followed by the deployment of the antenna and solar panels. The spacecraft then sends its first beacon for ground station connection. During this state, early subsystem setup takes place.

Mission Software Architecture

Initialization state is entered after deploying from the launcher. Detumbling executes until reaching stable body rates, followed by the deployment of the antenna and solar panels. The spacecraft then sends its first beacon for ground station connection. During this state, early subsystem setup takes place.

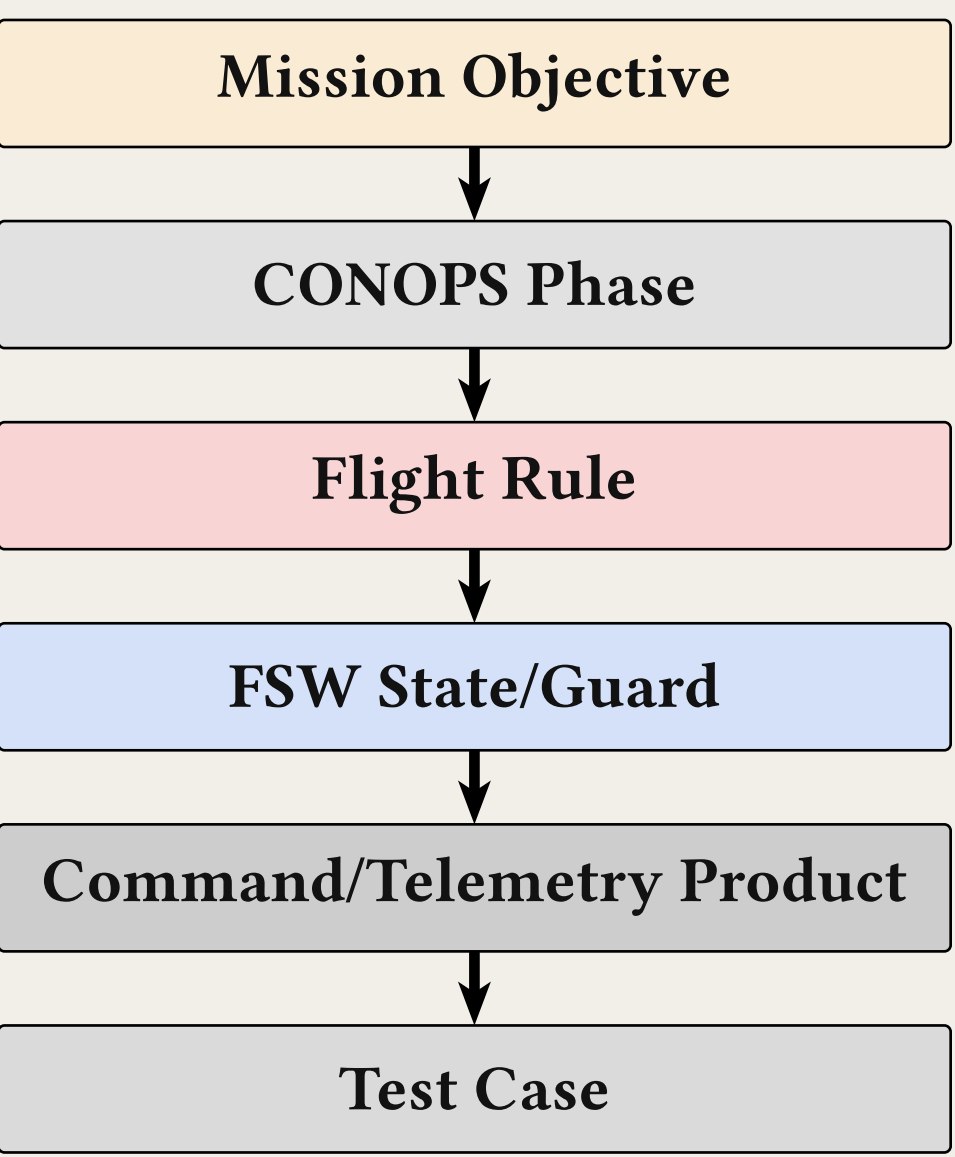
Safe state is entered whenever the spacecraft is undergoing degraded health. The spacecraft operates with minimum required functionality regarding power, communication, and recoverability. Once the spacecraft returns to acceptable conditions, a transition is made into idle. Commands can still be queued, but execution is deferred until transitioning into the appropriate state.

Idle state is the nominal state, performing health checks, peripheral polling, memory scrubbing, and ground-contact monitoring. Experiment commands can be queued, but execution is delayed until guards for the experiment state are satisfied.

Experiment state is entered upon satisfying requirements for mission phase, resource state, payload readiness, and data-storage. This state contains setup, execution, and closeout substates. Once operator inputs for OHP variant, sunlight phase, and heater power level are verified, and once the temperature, power margin, attitude stability, and storage availability meet requirements, a transition to execution is made for thermal data collection. At closeout, data is stored and a transition to idle is made.

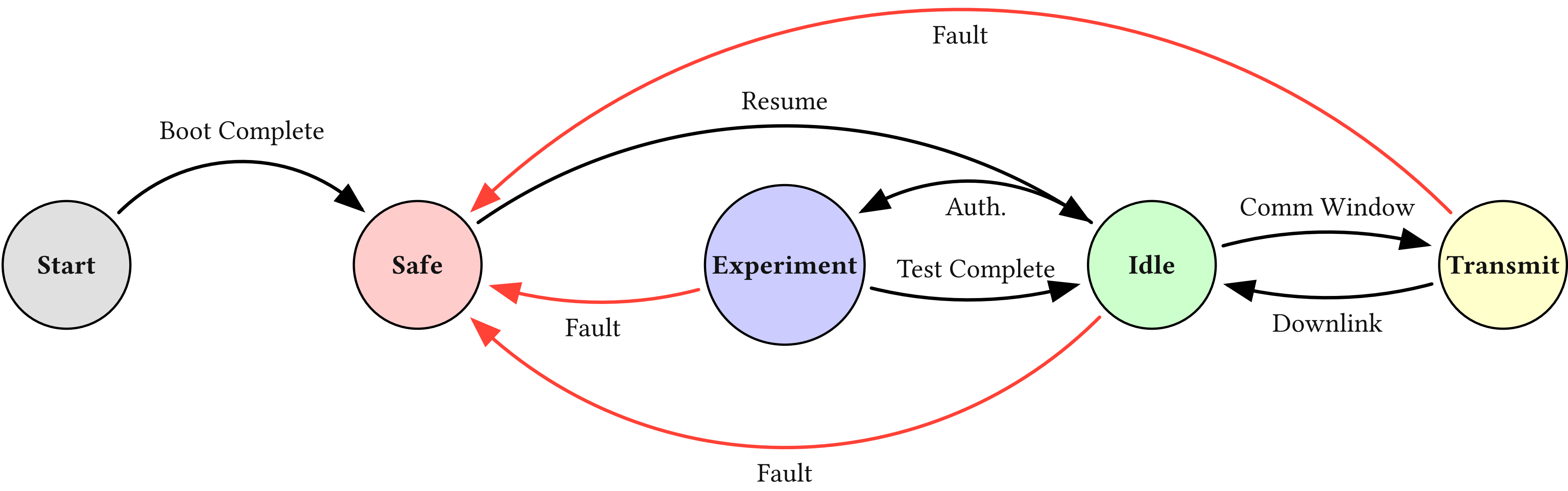
Transmit state

A transition to transmit state is made from idle once ground-station contact is made to downlink mission data and receive commands. Lost contact causes a transition to idle.

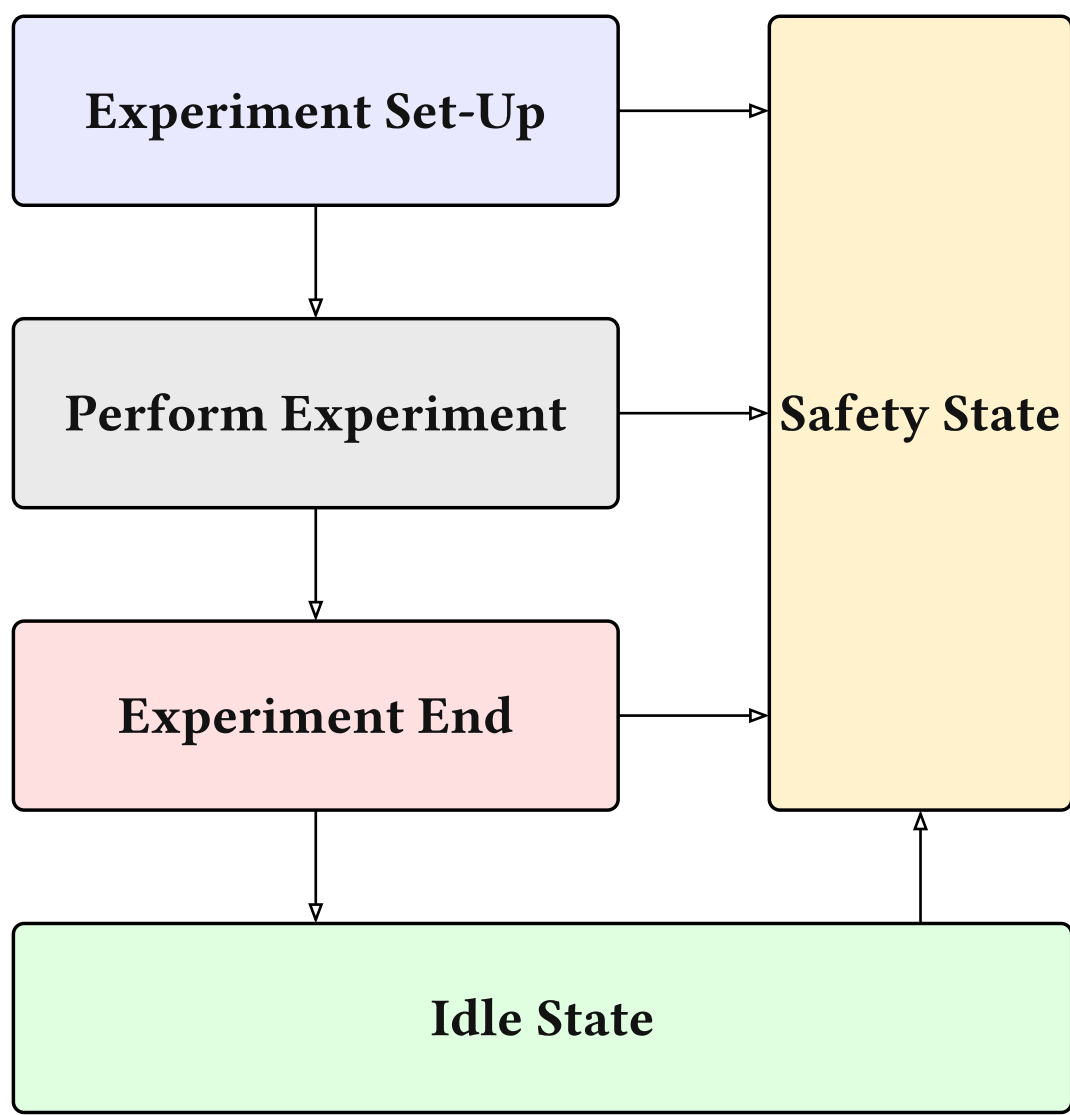


Mission Software Behavioral Logic

CubeSTEP software behavior is organized around a constrained state machine. Each state defines permitted commands, telemetry expectations, subsystem authority, and allowed transitions.



Transitions evaluate spacecraft health, power margin, thermal limits, communications availability, payload readiness, and mission phase. Commands that violate a state's authority boundary are rejected and produce an explanatory event for ground operators.



Acknowledgements

We thank the CubeStep-Cerberus team, the Flight Software and Mission Design and Operations subteams, and our advisors Navid Nakhjiri for their technical guidance and project support. We also acknowledge NASA JPL's F Prime project and the open-source contributors whose work enabled this embedded flight software development effort.

References

- [1] EnduroSat, "Onboard Computer I: OBCI." 2026.
- [2] Amazon Web Services, "FreeRTOS Kernel." 2026.
- [3] NASA Jet Propulsion Laboratory, "F Prime: A Flight-Proven, Multi-Platform, Open-Source Flight Software Framework." 2026.
- [4] STMicroelectronics, "STM32CubeH7: STM32Cube MCU Package for STM32H7 Series." 2026.